

Simplicity Goes Far: Auditing Prompt Optimizers with Demonstration-Conditioned Prompt Search

Anonymous Author(s)

Anonymous Institute

Abstract. Automatic prompt optimization (APO) is growing more elaborate, stacking policy-gradient updates, trace reflection, and evolutionary search. These mechanisms can raise peak accuracy but demand large rollout and token budgets, and it remains unclear how much of the gain comes from the added machinery rather than the generate-and-select step they all share. We isolate that step as **Demonstration-Conditioned Prompt Search** (DCPS), repurposing the Automatic Prompt Engineer (APE) generate-and-select recipe as a mechanism-attribution control: it freezes the generator, conditions it on randomly sampled demonstrations, scores candidates on validation data, and keeps the best—no learned update, reflection, or evolution. Instantiated in two optimizer families, DCPS replaces Proximal Policy Optimization (PPO) in StablePrompt, matching classification accuracy within 0.5 pp at roughly one-third the cost; in DSPy-based compound AI systems, under the reported budgets, it matches GEPA’s macro accuracy on GPT-4.1-mini at the highest relative error reduction (+26.23% vs. GEPA’s +24.15%), using $2.2\times$ fewer rollouts and $1.5\times$ lower API cost per task on average. The audit also locates where heavier mechanisms still pay off: open-ended generation and multi-hop tasks requiring trace-level error attribution. We recommend benchmarking prompt optimizers against generate-and-validate controls, reporting rollout, token, and dollar costs alongside accuracy. Code will be released upon acceptance.

Keywords: Prompt Optimization · Mechanism Audit · Prompt Search · Cost Analysis

1 Introduction

Automatic prompt optimization (APO) has become increasingly elaborate. Reinforcement learning (RL) methods [3,12] often rely on long rollout streams and policy-optimization pipelines, while reflection-based methods [1] use natural-language feedback to diagnose failures and rewrite prompts. These mechanisms can improve peak accuracy, but they also increase compute and API costs and make attribution harder. When a prompt optimizer works, is the gain coming from policy updates, reflection, and Pareto-style evolution, or from a simpler process of proposing prompts and selecting the best one on validation data?

To answer this question, we study **DCPS (Demonstration-Conditioned Prompt Search)**, a deliberately minimal generate-and-validate control built from the Automatic Prompt Engineer (APE) generate-and-select pattern [27]. Unlike APE, which introduced this pattern as a standalone prompt-search algorithm, DCPS repurposes it as a control for mechanism attribution: the same proposal-and-selection loop is inserted into existing optimizer settings to probe what policy updates, reflection, merging, or Bayesian search add beyond validation selection. Demonstrations condition the generator to propose prompts but are not automatically carried into the final target-model prompt; DCPS is therefore neither a few-shot example selector nor a new optimizer, but a diagnostic tool whose contribution lies in the audit conclusions it enables.

We apply this control in two representative APO settings, auditing each mechanism where it is most established: PPO on StablePrompt’s single-prompt benchmarks, and reflection and Bayesian search on GEPA-style compound systems. The template is instantiated twice, differing only in the selection rule. **StablePrompt-DCPS** instantiates DCPS inside StablePrompt [12], freezing the prompt generator and removing PPO updates while preserving the original demonstration-conditioned proposal and Top- K selection. **DCPS-Compound** instantiates the same control on GEPA-style compound AI systems [1], keeping the task setting and rollout evaluation but removing trace reflection, prompt mutation, merge, and Pareto evolution. We further report the Relative Cost-Efficiency Index (RCEI) as an auxiliary view of the token–performance trade-off.

Contributions.

1. **A reusable audit methodology.** We formalize DCPS as a four-stage control template and instantiate it inside two structurally different optimizer families—an RL-based pipeline (StablePrompt) and compound AI systems (GEPA and the heavy-budget MIPROv2 configuration)—to probe how much individual optimizer mechanisms add beyond validation selection.
2. **A diagnostic boundary for optimizer complexity.** Across both settings, proposal and selection alone recover most gains on structured tasks at lower measured cost, while policy updates and reflection retain genuine value on open-ended generation and multi-hop reasoning—locating a task-dependent boundary for when optimizer overhead is justified.
3. **Cost-aware evaluation.** We report accuracy jointly with rollout counts, token usage, and dollar costs, and use RCEI as an auxiliary cost-efficiency view, giving a replicable protocol for comparing optimizers under their actual budgets.

2 Related Work

Generate-and-select controls. Instruction Induction [7] and APE [27] elicit natural-language instructions from demonstrations and select them by task performance. Notably, APE itself reports that its optional iterative (Monte Carlo)

search saturates within a few rounds and is disabled by default for diminishing returns—an early single-mechanism hint that added search may contribute little once proposal and selection are in place. DCPS inherits this structure but changes its role: rather than another standalone optimizer, it serves as a controlled ablation for existing pipelines, systematizing that hint across policy updates, reflection, and evolution. A gain a heavier optimizer fails to show over such a control provides little evidence that it stems from those mechanisms.

RL updates and reflection loops. Existing APO methods [19] broadly split into two paradigms. RL-based optimizers train an auxiliary generator to reshape discrete token distributions; examples include RLPrompt, StablePrompt, PRewrite, and MORL-Prompt [3,12,11,8]. Much of this engineering stabilizes the policy update: StablePrompt, for instance, introduces an adaptive PPO variant because vanilla PPO collapses on discrete prompt tokens. Reflection-based methods, including ProTeGi [16], TextGrad [26], PromptAgent [22], and GEPA [1], instead use an LLM to read execution traces and propose textual edits, often with upper-confidence-bound (UCB), beam or tree search, or Pareto-frontier selection.

Compound AI systems. Compound systems chain multiple language modules under shared control flow, closer to production than single-prompt benchmarks. DSPy [10] treats the program as a compiler target for joint prompt optimization; MIPROv2 [15] adds Bayesian prompt search, while GEPA [1] adds reflective rewriting and Pareto-frontier merging over per-module traces, often at thousands of rollouts. We use DCPS-Compound on the same tasks as a reflection-free reference point.

Mechanism controls and cost. Recent analyses of reinforcement learning from human feedback (RLHF) and reinforcement learning from verifiable rewards (RLVR) suggest some gains attributed to policy updates may already be reachable through sampling and selection: Yue et al. [25] use $\text{pass}@k$ to argue RLVR mainly sharpens the base model’s sampling distribution, while RAFT [4] shows supervised fine-tuning on reward-ranked best-of- N samples can approach PPO-style alignment. DCPS brings the same control to prompt optimization. Because APO papers often report accuracy despite large differences in rollouts and dollar costs, RCEI complements the audit by making marginal return per token explicit.

3 Demonstration-Conditioned Prompt Search

3.1 Task Definition

Following the definition of compound AI systems [10,1], we write the target system as $\Phi = (M, C, \mathcal{X}, \mathcal{Y})$, where $M = \langle M_1, \dots, M_{|M|} \rangle$ is a sequence of language modules, C is the control flow, and $\mathcal{X}, \mathcal{Y}$ are the global input and output schemas. Each module $M_i = (\pi_i, \theta_i, \mathcal{X}_i, \mathcal{Y}_i)$ contains a prompt π_i , fixed parameters θ_i , and local schemas. Let Π_Φ collect all module prompts and Θ_Φ the fixed target-model weights. Given a task distribution $\mathcal{T}$ over instances (x, m) and a

metric $\mu \in [0, 1]$, prompt optimization searches under rollout budget B for:

$$\Pi_{\Phi}^* = \arg \max_{\Pi_{\Phi}} \mathbb{E}_{(x,m) \sim \mathcal{T}} [\mu(\Phi(x; \Pi_{\Phi}, \Theta_{\Phi}), m)] \quad (1)$$

A rollout is one evaluation call to Φ under a candidate prompt configuration. This covers both single-prompt benchmarks ($|M| = 1$) and compound pipelines (where Π_{Φ} is jointly optimized across modules).

3.2 DCPS Pipeline

DCPS uses one optimization signal: the validation performance of prompts generated from randomly sampled demonstrations. The demonstrations shape the generator’s proposal but do not by themselves become part of the final inference prompt. Given a target system Φ , a training set D_{train} , a fixed validation subset $D_{\text{val}}^{\text{sub}}$, a frozen generator $p_{\theta_G^{(0)}}$, and metric μ , DCPS iterates four stages on the candidate prompt set:

- **Demonstration sampling $\mathcal{S}$.** Sample random k -shot examples from D_{train} and format them into a meta-prompt $\mathcal{I}^{(t)}$ for the generator.
- **Candidate generation $\mathcal{G}_{\theta_G^{(0)}}$.** Draw candidate prompts $\Pi^{(t)} \sim p_{\theta_G^{(0)}}(\cdot | \mathcal{I}^{(t)})$ from the frozen generator, with no parameter update, trace-based reflection, or rewrite loop.
- **Evaluation $\mathcal{E}$.** Score each generated prompt on the fixed validation subset $D_{\text{val}}^{\text{sub}}$ under μ .
- **Selection $\mathcal{F}$.** Retain the best candidate(s): greedy replacement in compound runs, or Top- K retention in StablePrompt-DCPS.

The resulting control operator is

$$\mathcal{O}_{\text{DCPS}}(\mathcal{T}, \Phi, B) = \mathcal{F} \circ \mathcal{E} \circ \mathcal{G}_{\theta_G^{(0)}} \circ \mathcal{S}. \quad (2)$$

At evaluation time the target system receives the generated prompt $\Pi^{(t)}$; the sampled demonstrations affect it only through the generator’s meta-prompt, unless the generator copies them into its output. This separates DCPS from few-shot example selection and from few-shot in-context evaluation, while making the borrowed component explicit: APE-style demonstration-conditioned proposal coupled to score-based selection [27], without policy updates, trace reflection, prompt mutation, or merge operations.

3.3 Two DCPS Audits

We instantiate the DCPS control template in two settings, comparing against StablePrompt, GEPA, and MIPROv2-Heavy (the heavy-budget MIPROv2 configuration) as three representative sources of complexity in modern APO: policy-gradient updates, trace-reflection with prompt evolution, and Bayesian search over instruction–demonstration programs. The selector is matched to each host:

StablePrompt-DCPS keeps StablePrompt’s Top- K queue and training-batch rewards, while DCPS-Compound uses greedy validation replacement. These serve as a *policy-update audit* of StablePrompt and a *compound-system audit* against GEPA-style reflection and MIPROv2-style Bayesian search (Algorithms 1 and 2).

StablePrompt-DCPS audit. StablePrompt [12] already contains the proposal stage DCPS isolates: an agent generator receives randomly sampled examples in a meta-prompt, proposes candidates, and is then updated with PPO-style rewards. **StablePrompt-DCPS** (Algorithm 1) freezes the initial generator weights $\theta_G^{(0)}$ and removes PPO updates, while keeping random k -shot sampling, the original reward computation, and Top- K retention. It also inherits StablePrompt’s data protocol—drawing demonstrations from $D_{\text{val}}^{\text{demo}}$ and scoring on training batches, reversing the default DCPS roles of §3.2—applied identically to PPO and DCPS, so the comparison is unaffected. The gap between StablePrompt-PPO and StablePrompt-DCPS estimates what the policy update adds beyond proposal and selection.

DCPS-Compound for compound AI systems. For compound AI systems, DCPS-Compound follows the task and rollout protocol used for GEPA-family and MIPROv2-Heavy optimizers [1,15] (Algorithm 2), auditing what trace reflection, Pareto selection, and Bayesian instruction–demonstration search add beyond proposal and validation. Each iteration samples demonstrations from D_{train} , queries the frozen generator, evaluates on $D_{\text{val}}^{\text{sub}}$, and replaces the incumbent when validation improves.

4 Experimental Setup

We run two evaluations: §4.1 evaluates StablePrompt-DCPS on the original StablePrompt single-prompt benchmarks, and §4.2 evaluates DCPS-Compound on compound AI systems against GEPA, GEPA-MERGE (GEPA with prompt merging), and MIPROv2-Heavy. All runs reuse the original StablePrompt [12], DSPy [10], and GEPA [1] implementations with published default hyperparameters; token and USD costs are measured from Weights & Biases (WandB) summaries, local logs, and API-provider activity records. Full configurations and artifacts will be released with the code.

4.1 StablePrompt-DCPS Ablation

We adopt the full StablePrompt configuration of Kwon et al. [12]: both target and agent models are **gemma-1.1-7B-it** [5], evaluated on the same three task families: (1) **text classification** (6 GLUE/SuperGLUE natural-language-understanding subsets [21,20]), (2) **multi-task QA** (57-subject Massive Multitask Language Understanding (MMLU) [6]), and (3) **instruction induction**

Algorithm 1 StablePrompt-DCPS

Require: Training set D_{train} , validation set $D_{\text{val}}^{\text{demo}}$, frozen generator $p_{\theta_G^{(0)}}$, target system Φ , metric μ , epochs T , candidates per batch N , few-shot size k , queue capacity K

- 1: $\mathcal{P} \leftarrow \emptyset$, $t \leftarrow 0$
- 2: **for** $e = 1$ to T **do**
- 3: **for** $B_j \in \text{Batches}(D_{\text{train}})$ **do**
- 4: $t \leftarrow t + 1$
- 5: $E^{(t)} \leftarrow \text{Sample}(D_{\text{val}}^{\text{demo}}, k) \{S\}$
- 6: $\mathcal{I}^{(t)} \leftarrow \text{FmtMeta}(E^{(t)})$
- 7: $\{\Pi_i^{(t)}\}_{i=1}^N \sim p_{\theta_G^{(0)}}(\cdot | \mathcal{I}^{(t)}) \{G\}$
- 8: **for** $i = 1$ to N **do**
- 9: $s_i^{(t)} \leftarrow \text{Eval}(\Pi_i^{(t)}, B_j) \{E\}$
- 10: $c_i^{(t)} \leftarrow (\Pi_i^{(t)}, s_i^{(t)})$
- 11: $\mathcal{P} \leftarrow \text{TopK}(\mathcal{P} \cup \{c_i^{(t)}\}, K) \{F\}$
- 12: **end for**
- 13: **end for**
- 14: **end for**
- 15: **return** $\mathcal{P}$

Algorithm 2 DCPS-Compound

Require: Training set D_{train} , validation set D_{val} , frozen generator $p_{\theta_G^{(0)}}$, target system Φ , metric μ , iterations I , few-shot size k , val. subset n_v

- 1: $\Pi^* \leftarrow \text{null}$, $s^* \leftarrow -\infty$; $D_{\text{val}}^{\text{sub}} \leftarrow \text{Sample}(D_{\text{val}}, n_v)$
- 2: **for** $t = 1$ to I **do**
- 3: $E^{(t)} \leftarrow \text{Sample}(D_{\text{train}}, k) \{S\}$
- 4: $\mathcal{I}^{(t)} \leftarrow \text{FmtMeta}(E^{(t)})$
- 5: $\Pi^{(t)} \sim p_{\theta_G^{(0)}}(\cdot | \mathcal{I}^{(t)}) \{G\}$
- 6: $s^{(t)} \leftarrow \text{Eval}(\Pi^{(t)}, D_{\text{val}}^{\text{sub}}) \{E\}$
- 7: **if** $s^{(t)} > s^*$ **then**
- 8: $\Pi^* \leftarrow \Pi^{(t)}$; $s^* \leftarrow s^{(t)} \{F\}$
- 9: **end if**
- 10: **end for**
- 11: **return** Π^*

Fig. 1. Left: StablePrompt-DCPS. Right: DCPS-Compound. $\text{FmtMeta}(E)$ formats the k -shot examples into a meta-prompt for the generator; $\text{Eval}(\Pi, D) = \frac{1}{|D|} \sum_{(x,m) \in D} \mu(\Phi(x; \Pi, \Theta_\Phi), m)$. In StablePrompt-DCPS, $D_{\text{val}}^{\text{demo}}$ supplies only demonstrations and rewards are computed on training batches B_j .

& generation (24 Instruction Induction (II) [7] subsets + 18 Big-Bench Instruction Induction (BBII) [27] subsets, split into BBII-TC (text-classification subsets) and BBII-Gen (generation subsets)), with the original scoring protocols. The DCPS variant removes gradient computation and PPO optimizer states, so it runs inference-only on a single NVIDIA A40 48 GB, whereas StablePrompt-PPO follows the original A100 80 GB setup; cost tables report measured wall-clock time at both rates. In reproduction we fix a train/test overlap in II and missing batch weights in StablePrompt’s Softmax-difference aggregation metric. Following StablePrompt’s protocol, we report the mean over three seeds of the best test score among the Top-5 prompts ranked by training reward. This protocol is optimistic in absolute terms because final Top-5 selection touches test scores; we apply it identically to PPO and DCPS, so the PPO–DCPS comparison our audit targets is unaffected.

4.2 Compound AI System Optimization Setup

We use Qwen3-8B [18] and GPT-4.1-mini [14] as both the target system and the prompt generator. The six benchmarks cover multi-step retrieval tasks, HotpotQA [24] and HoVer [9], plus single- or two-stage reasoning and control tasks:

IFBench (verifiable instruction following) [17], PUPA (privacy delegation from PAPILLON) [13], AIME-2025 [2], and LiveBench-Math [23]. DCPS-Compound uses task-specific rollout budgets reported with the results and validation subset sizes $n_v \in [15, 50]$ depending on task validation size.

4.3 Cost and Efficiency Metrics

We report accuracy together with rollout budgets and measured cost. For cost-normalized comparison, we use relative error reduction (RER), which normalizes gains by the remaining headroom over the unoptimized prompt:

$$\text{RER}(\mathcal{M} \parallel \mathcal{B}) = \frac{\mathcal{M} - \mathcal{B}}{100\% - \mathcal{B}} \times 100\%, \quad (3)$$

where $\mathcal{B}$ is the unoptimized prompt score and $\mathcal{M}$ is the optimized method score. Cost is measured by price-weighted token use across optimization and final test evaluation:

$$T = T_{\text{optimizer}} + T_{\text{test}}, \quad T_{(\cdot)} = T_{\text{in}}^{(\cdot)} + 4T_{\text{out}}^{(\cdot)}, \quad (4)$$

using the approximate GPT-4.1-mini output-to-input price ratio. RCEI reports relative error reduction per log cost multiplier:

$$\text{RCEI}(\mathcal{M} \parallel \mathcal{B}) = \frac{\text{RER}(\mathcal{M} \parallel \mathcal{B})}{\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})}. \quad (5)$$

Since optimization always consumes tokens, $T_{\mathcal{M}} > T_{\mathcal{B}}$ and the denominator is strictly positive. RCEI is an auxiliary view, reported with accuracy, rollouts, and dollar cost; macro RER and RCEI are arithmetic means over per-task values.

5 Main Results and Diagnostic Findings

We report accuracy alongside rollout and cost accounting for both audits. §5.1 uses StablePrompt-DCPS to ask what PPO adds over the frozen-generator control, and §5.2 asks the same of reflection and Bayesian search. The two differ in granularity: the StablePrompt study is a single-variable ablation, while the compound-system study is an accuracy–cost control comparison, not a budget-matched ranking.

5.1 StablePrompt-PPO vs StablePrompt-DCPS

Table 1 shows PPO updates have limited marginal value on the classification tasks, though they keep an edge on MMLU and BBII-Gen. On GLUE/SuperGLUE [21,20] and BBII-TC [27], DCPS and PPO do not differ significantly ($|\Delta| \leq 0.4$ pp), and Figure 2(a) shows DCPS tracking PPO throughout training on SST-2.

The picture changes on tasks where the desired behavior is less template-like. PPO leads by +2.3 pp on BBII-Gen [27] and +1.8 pp on MMLU [6], suggesting policy updates still help when the target behavior is hard to reach by random

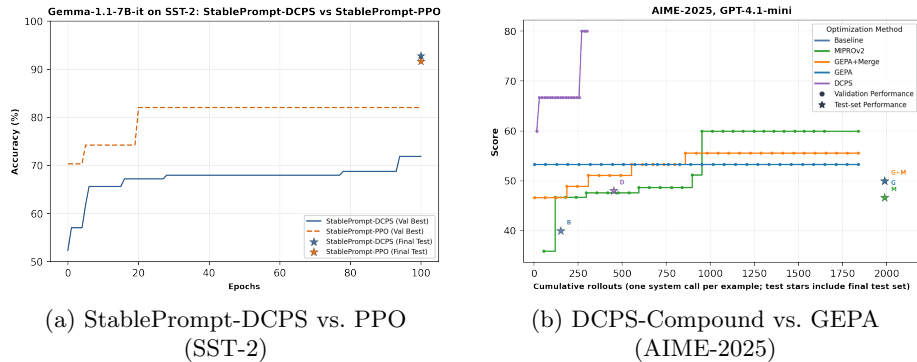


Fig. 2. Trajectory and anytime views for both audits. (a) Running-best SST-2 validation accuracy for StablePrompt-DCPS vs. PPO on `gemma-1.1-7B-it`; stars mark the final test score at epoch 100. (b) Test accuracy as cumulative rollouts increase on GPT-4.1-mini AIME-2025.

Table 1. StablePrompt-PPO vs. StablePrompt-DCPS on `gemma-1.1-7B-it`: accuracy ablation and measured compute cost. Scores are macro-averages over subsets; Δ (PPO–DCPS), 95% CI, and Cohen’s d are over per-subset differences (vs. PPO@30 epochs for extended II runs). DCPS runs inference-only on an A40 ($\approx$ \$0.40/hr) vs. PPO’s A100 ($\approx$ \$1.50/hr); wall-clock hours give a rate-independent view.

Task Family	PPO	DCPS	Δ	95% CI	d	DCPS time/cost	PPO time/cost	Note
GLUE/SuperGLUE	76.7	77.1	−0.4	[−2.30, +1.97]	−0.14	4.83h/\$1.93	3.97h/\$5.96	comparable accuracy
BBII-TC	57.6	57.2	+0.4	[−0.68, +1.63]	+0.20	3.38h/\$1.35	3.17h/\$4.76	comparable accuracy
BBII-Gen	63.0	60.7	+2.3	[+0.15, +4.35]	+0.79	2.66h/\$1.06	1.84h/\$2.76	PPO advantage
II @30 epochs	48.7	44.3	+4.4	[+0.15, +8.70]	+0.41	2.76h/\$1.10	6.65h/\$9.98	PPO advantage at equal epochs
II @100 epochs	—	48.6	+0.1	[−2.67, +2.63]	+0.01	14.74h/\$5.90	—	gap closed by longer search
MMLU	55.9	54.1	+1.8	[+0.99, +2.74]	+0.53	6.58h/\$2.63	6.38h/\$9.57	PPO advantage

conditioning. On II [7], extending DCPS to 100 epochs closes the gap with PPO at 30 epochs (+0.1 pp) at lower measured cost (and a further extension to 200 epochs holds at 49.1, $\Delta = -0.4$); since PPO is not re-run at matching epochs, this shows extra search budget can substitute for the policy update rather than that DCPS strictly dominates. Where accuracy is matched, removing the policy update drops gradient computation and optimizer states, enabling inference-only runs on a cheaper GPU tier: measured single-run cost falls from \$4.76–\$5.96 (A100) to \$1.35–\$1.93 (A40), roughly $3\times$ under our rates.

5.2 DCPS on Compound AI Systems

Table 2(a) summarizes the cost–quality trade-off. At under half of GEPA’s average rollout budget (1,761 vs. 3,936) and a lower API cost, DCPS-Compound attains both the highest Macro RER (+26.23%) and the highest Macro RCEI among optimized methods, while matching GEPA’s absolute macro accuracy on GPT-4.1-mini (61.14, single seed; see §6.2). MIPROv2-Heavy [15], run at GEPA’s full budget, trails on both relative error reduction and cost-efficiency.

Table 2. Compound-system results. **(a)** Cost-performance summary on GPT-4.1-mini, macro-averaged across the six benchmarks ($\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})$ with T as in §4.3). **(b)** Task-level accuracy on Qwen3-8B and GPT-4.1-mini; macro-average is the unweighted mean over six benchmarks, and the bottom block lists per-task rollout budgets (shared across backbones). Each method runs at a single budget (see text).

(a) Cost–performance summary (GPT-4.1-mini, macro-averaged)						
Method ($\mathcal{M}$)	RER	$T_{\mathcal{M}}$	$\log_2(T_{\mathcal{M}}/T_{\mathcal{B}})$	RCEI	Rollouts	Cost
Baseline (unoptimized prompt, $\mathcal{B}$)	0	1.06	0	–	–	\$0.43
MIPROv2-Heavy	+12.70%	27.11	4.67	+2.66	3,936	\$10.84
GEPA	+24.15%	28.36	4.73	+5.30	3,936	\$11.34
GEPA-MERGE	+24.87%	24.07	4.50	+5.90	3,936	\$9.63
DCPS-Compound	+ 26.23%	19.26	3.85	+7.20	1,761	\$7.70

(b) Task-level accuracy								
Backbone	Method	HotpotQA	IFBench	HoVer	PUPA	AIME-2025	LiveBench-Math	Macro-Avg
Qwen3-8B	Baseline	40.33	38.61	33.67	81.55	47.33	65.01	51.08
	MIPROv2-Heavy	58.67	38.61	44.00	78.67	58.00	67.46	57.57
	GEPA	63.33	40.65	53.33	87.57	59.33	70.57	62.46
	GEPA-MERGE	61.33	29.08	52.00	84.81	62.67	66.28	59.36
	DCPS-Compound	61.33	43.88	56.67	90.02	55.33	65.08	62.05
GPT-4.1-mini	Baseline	36.10	48.13	40.33	80.81	40.00	55.80	50.20
	MIPROv2-Heavy	55.00	51.19	47.33	83.59	46.67	57.30	56.85
	GEPA	65.00	49.83	49.67	90.10	48.00	64.21	61.14
	GEPA-MERGE	63.33	49.49	49.00	93.52	46.67	61.14	60.53
	DCPS-Compound	59.00	51.53	54.67	94.09	48.00	59.52	61.14
<i>Total optimization budget (# rollouts)</i>								
	GEPA/GEPA-MERGE/MIPROv2-Heavy	6,871	3,593	7,051	2,426	1,839	1,839	3,936
	DCPS-Compound	1,020	620	7,068	920	320	620	1,761

Each method is evaluated at a single reported rollout budget: GEPA-family methods at their published defaults and DCPS-Compound at the budgets in Table 2. HoVer is the exception: a small-budget run matching the other tasks was weak, so the main table reports the GEPA-aligned extended run. §5.3 reports a budget-sensitivity analysis on three tasks, leaving a full sweep for future work.

Table 2(b) reveals the underlying mechanism boundary. DCPS-Compound is strongest on single- or two-stage tasks that hinge on precise instruction following (IFBench, PUPA), and on HoVer once adequately budgeted (§5.3). GEPA’s advantages concentrate on reasoning-heavy tasks: HotpotQA (+6.00 pp on GPT-4.1-mini), LiveBench-Math (both backbones), and Qwen AIME-2025. DCPS offsets these on IFBench, PUPA, and budgeted HoVer, so the GPT-4.1-mini macro tie reflects offsetting per-task results, not method equivalence. GEPA-MERGE, by contrast, falls below even the unoptimized Qwen IFBench baseline, indicating that crossover-based prompt merging can be unstable on strict-format tasks. On GPT-4.1-mini AIME-2025, Figure 2(b) shows DCPS-Compound reaching its final accuracy early and matching GEPA at a smaller rollout budget.

5.3 Budget Sensitivity and the Generalization Gap

The per-task budgets in Table 2 raise a natural question: when does more rollout budget help, and when does it merely pressure the selector? We extend DCPS on

three diagnostic tasks under the same validation-only selection rule: HotpotQA (to 6,120 rollouts) and HoVer toward GEPA-scale budgets, and LiveBench-Math to 1,860 rollouts. These runs are not new leaderboard entries; they test whether validation-selected prompts continue to generalize as the candidate pool grows. For HoVer, the main table reports the extended DCPS run because the initial small-budget run (620 rollouts, matching the other small-budget tasks) was clearly under-budgeted (44.00 on Qwen and 45.67 on GPT).

The probes separate three regimes. **HoVer** is mainly a coverage bottleneck: with GEPA-scale sampling, DCPS surpasses GEPA on both backbones while keeping validation–test gaps small (≤ 3.3 pp), so the initial GEPA advantage was a budget artifact. **HotpotQA** is different: extending DCPS to 6,120 rollouts raises GPT-4.1-mini from 59.00 to 63.67 (narrowing the gap to GEPA’s 65.00 from 6.00 to 1.33 pp) but leaves Qwen3-8B essentially flat (61.33 $\rightarrow$ 60.33, vs. GEPA’s 63.33). Extra rollouts thus narrow the gap on GPT-4.1-mini but do not remove GEPA’s edge on either backbone, suggesting that trace-level error attribution remains useful when failures must be localized across retrieval and reasoning steps.

LiveBench-Math is the cautionary case. More rollouts raise validation accuracy but lower held-out accuracy, producing large generalization gaps (20.95 and 13.50 pp), a single-seed signal consistent with selection overfitting. As the candidate pool grows, DCPS is more likely to find prompts whose validation scores are inflated by quirks of a small validation subset. Thus rollout budget is not only compute; it is also statistical pressure on the selector.

6 Discussion

6.1 Applicability Boundaries from Optimization Signal to Pipeline Complexity

The results point to a practical dichotomy: heavy mechanisms help mainly when the task requires information that proposal and validation cannot reliably expose. On tasks bottlenecked by output formatting or short-context instruction following, random demonstrations give the frozen generator enough signal, so PPO and reflective rewriting add little relative to their cost. When the bottleneck shifts to open-ended generation, logical routing, or cascading failures, DCPS hits a ceiling that policy updates and reflection can lift. Heavy optimizers are thus better seen as specialized tools whose overhead must be justified against a simple control.

6.2 Limitations and Empirical Boundaries

The conclusions are subject to several limitations. **Non-identical settings:** the StablePrompt audit uses the original `gemma-1.1-7B-it` setup while the compound runs use Qwen3-8B and GPT-4.1-mini to match GEPA, so each part audits its mechanism in its native setting rather than under one fixed backbone. **Single-budget, single-seed:** the headline compares each optimizer at

one budget—GEPA-family methods at published defaults, DCPS at task-specific budgets that are not exhaustively swept—on small single-seed test sets. Our claim is therefore an *existence proof* that a simple control can reach competitive accuracy–cost points, not a budget-matched ranking; generalization-gap figures are indicative and multi-seed replication would strengthen them. **Optimization capacity:** lacking trace-based rewriting and module-level attribution, DCPS is limited when cascading errors require localized diagnosis. **Cost metrics:** RCEI depends on token pricing, output weighting, and log availability, and GPU costs depend on rental rates, so rankings may shift under other price structures. **Task coverage:** code generation, summarization, tool use, and longer-context interactive tasks remain untested.

7 Conclusion

We asked how much of the gain credited to complex prompt-optimization mechanisms survives once the optimizer is stripped down to proposal and validation selection. The answer, across StablePrompt and GEPA-style compound systems, is most of it: on structured tasks DCPS recovers nearly all the gain at a fraction of the cost, faltering only on reasoning-heavy tasks and when validation-only selection overfits a small validation set. The methodological takeaway is simple—future optimizers should be measured against a strong generate-and-validate control, with cost reported alongside accuracy.

References

1. Agrawal, L.A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M.J., Jiang, M., Potts, C., Sen, K., Dimakis, A.G., Stoica, I., Klein, D., Zaharia, M., Khattab, O.: GEPA: Reflective prompt evolution can outperform reinforcement learning. In: ICLR (2026), oral
2. Balunovic, M., Dekoninck, J., Petrov, I., Jovanovic, N., Vechev, M.: MathArena: Evaluating LLMs on uncontaminated math competitions. <https://matharena.ai/> (2025)
3. Deng, M., Wang, J., Hsieh, C.P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E.P., Hu, Z.: RLPrompt: Optimizing discrete text prompts with reinforcement learning. In: EMNLP (2022)
4. Dong, H., Xiong, W., Goyal, D., Zhang, Y., Chow, W., Pan, R., Diao, S., Zhang, J., Shum, K., Zhang, T.: RAFT: Reward rAnked FineTuning for generative foundation model alignment. Transactions on Machine Learning Research (TMLR) (2023)
5. Gemma Team: Gemma: Open models based on Gemini research and technology. arXiv:2403.08295 (2024)
6. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., Steinhardt, J.: Measuring massive multitask language understanding. In: ICLR (2021)
7. Honovich, O., Shaham, U., Bowman, S.R., Levy, O.: Instruction induction: From few examples to natural language task descriptions. In: ACL (2023)
8. Jafari, Y., Mekala, D., Yu, R., Berg-Kirkpatrick, T.: MORL-Prompt: An empirical analysis of multi-objective reinforcement learning for discrete prompt optimization. In: Findings of EMNLP (2024)

9. Jiang, Y., Bordia, S., Zhong, Z., Dognin, C., Singh, M., Bansal, M.: HoVer: A dataset for many-hop fact extraction and claim verification. In: Findings of EMNLP (2020)
10. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Haq, S., et al.: DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In: ICLR (2024)
11. Kong, W., Hombaiah, S.A., Zhang, M., Mei, Q., Bendersky, M.: PRewrite: Prompt rewriting with reinforcement learning. In: ACL (2024)
12. Kwon, M., Kim, G., Kim, J., Lee, H., Kim, J.: StablePrompt: Automatic prompt tuning using reinforcement learning for large language model. In: EMNLP (2024)
13. Li, S., Raghuram, V.C., Khattab, O., Hirschberg, J., Yu, Z.: PAPILLON: Privacy preservation from internet-based and local language model ensembles. In: NAACL (2025)
14. OpenAI: GPT-4.1 series. <https://openai.com/index/gpt-4-1/> (2025)
15. Opsahl-Ong, K., Ryan, M.J., Purtell, J., Broman, D., Potts, C., Zaharia, M., Khattab, O.: Optimizing instructions and demonstrations for multi-stage language model programs. In: EMNLP (2024)
16. Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C., Zeng, M.: Automatic prompt optimization with “Gradient Descent” and beam search. In: EMNLP (2023)
17. Pyatkin, V., Malik, S., Graf, V., Ivison, H., Huang, S., Dasigi, P., Lambert, N., Hajishirzi, H.: Generalizing verifiable instruction following. In: NeurIPS Datasets and Benchmarks (2025)
18. Qwen Team: Qwen3 technical report. arXiv:2505.09388 (2025)
19. Ramnath, K., Zhou, K., Guan, S., Mishra, S.S., Qi, X., Shen, Z., Wang, S., Woo, S., Jeoung, S., Wang, Y., et al.: A systematic survey of automatic prompt optimization techniques. In: EMNLP (2025)
20. Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., et al.: SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In: NeurIPS. vol. 32 (2019)
21. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., Bowman, S.R.: GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv:1804.07461 (2018)
22. Wang, X., Li, C., Wang, Z., Bai, F., Luo, H., Zhang, J., Jojic, N., Xing, E.P., Hu, Z.: PromptAgent: Strategic planning with language models enables expert-level prompt optimization. In: ICLR (2024)
23. White, C., Dooley, S., Roberts, M., Pal, A., Feuer, B., Jain, S., Shwartz-Ziv, R., Jain, N., Saifullah, K., Dey, S., et al.: LiveBench: A challenging, contamination-limited LLM benchmark. arXiv:2406.19314 (2024)
24. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A dataset for diverse, explainable multi-hop question answering. In: EMNLP (2018)
25. Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., Huang, G.: Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In: NeurIPS (2025), oral
26. Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., Zou, J.: TextGrad: Automatic “Differentiation” via text. *Nature* **639**, 609–616 (2025)
27. Zhou, Y., Muresanu, A.I., Han, Z., Paster, K., Pitis, S., Chan, H., Ba, J.: Large Language Models are Human-Level Prompt Engineers. In: ICLR (2023)